

Slay The Lord

P박대원, I임찬진, P박경준
한국공학대학교 게임공학과

june25656@tukorea.ac.kr ckswlsdl0916@gmail.com pkj7323@tukorea.ac.kr

IOCP 기반 멀티플레이어 서버에서의 오픈소스 물리엔진을 활용한 서버 권위적 물리 시뮬레이션 및 최적화 연구

요 약

본 연구는 3D MO(Multiplayer Online) 액션 게임 환경에서 서버 권위(Server-Authoritative) 방식의 물리 시뮬레이션을 구현할 때 발생하는 CPU 연산 병목 및 멀티스레드 동기화 문제를 해결하기 위한 서버 아키텍처 및 최적화 기법을 제안한다. 이를 위해 윈도우즈 소켓 프로그래밍(IOCP) 기반의 비동기 네트워크 서버에 데이터 지향 설계(DOD) 및 멀티코어 환경에 최적화된 오픈소스 물리 엔진인 Jolt Physics를 통합하였다.

1. 서 론

1.1 연구 배경 및 목적

현대 3D 액션 게임에서 정교한 타격감과 판정을 위해서 물리엔진의 중요성이 커지고 있다. 이때 클라이언트-서버의 서버 권위적 모델을 사용한 물리연산을 통한 보안 취약점 방지와 유저 위치 동기화가 필수적이다. 물리엔진을 게임서버에서 직접 돌릴 경우 CPU병목이 발생하기 쉽으며 IOCP기반의 멀티스레딩 환경에서는 물리객체 접근시 발생하는 동기화 오버헤드의 문제점이 있다. 본 연구는 싱글스레드와 멀티스레드 환경 둘 다 지원하며 DOD와 멀티코어 환경에 최적화된 Jolt Physics 라이브러리를 서버에 통합하여 물리엔진 오버헤드를 최소한으로한 MO 액션 서버를 구현하는 것을 목적으로한다.

2. 개발 내용

2.1 Jolt Physics 구조탐색 및 Jolt Setup 구현

본 연구에서는 PhysicsManager를 싱글톤(Singleton) 구조로 설계하여 Jolt Physics의 팩토리 및 메모리 할당자를 전역 초기화하였다. 이후 각 Room 인스턴스가 생성될 때 독립적인 PhysicsSystem을 할당받아, 멀티스레드 환경에서도 물리 연산 공간이 완벽히 격리되도록 설계하였다.

특히, 연산 부하를 최소화하기 위해 Jolt Physics의 계층적 충돌 필터링(Hierarchical Collision Filtering) 시스템을 재선언하였다. ObjectLayer와 BroadPhaseLayer를 분리하여, 정적 지형(NON_MOVING)과 동적 객체(MOVING, NPC)의 충돌 검사 트리를 물리적으로 분리하였다. 이를 통해 지형 간의 무의미한 충돌 검사 연산을

브로드페이즈(Broad-Phase) 단계에서 $O(1)$ 에 가깝게 조기 차단 할 수 있다. 이때, NPC끼리의 충돌은 $O(n^2)$ 의 시간복잡도로 매우 큰 부하를 가져오기 때문에 ObjectLayerPairFilter 인터페이스를 확장하여 NPC들의 충돌을 Culling하여 대규모 객체가 스폰해도 CPU 점유율을 안정적유지 하기 위해 최적화하였다.

2.2 인스턴스 단위 물리공간 격리와 싱글스레드 최적화

우리 게임의 경우 MO게임이므로 각 인스턴스(room)은 IOCP 루프가 아닌 게임루프가 돌아가는 Logic_worker안에서 하나의 스레드안에서만 update가 일어난다. 따라서 이 장점을 살려서 각 물리공간을 인스턴스단위로 초기화해서 생성하고 JobSystemSingleThreaded를 사용하여 싱글스레드 최적화된 물리공간 생성하였다. 이때 각 로직스레드는 스레드 로컬한 20MB의 JPH::TempAllocatorImpl의 메모리를 개별적으로 가지고 자신의 모든 room인스턴스의 Physics Update에서 재사용함으로써 메모리 할당, 재사용 오버헤드를 제거했다. 이때 인스턴스들을 패킷처리와 Logic업데이트와 상관없이 물리 연산은 항상 16.66ms(60Hz)의 고정된 간격으로 실행하며 루프당 최대 물리 엔진연산횟수를 초과하면 초과 시간 누적분을 삭제하여 서버 복원력을 확보했다.

2.3 공간 분할(GridMap) 및 AOI 기반 동적 물리 LOD 스케줄링

하나의 인스턴스 내부의 NPC는 매우 많이 존재하는데 이때 모든 NPC의 물리 객체에

대해서 업데이트를 수행하면 CPU의 시간을 많이 잡아먹는 병목이된다. 이를 해결하기 위해서 플레이어의 시야리스트에서 착안하여 물리 LOD를 적용 하였다. NPC들을 저장하는 `std::unordered_set` `_activeNpcList`를 사용하여 플레이어 주변의 NPC들만 물리엔진 Update를 수행하고 멀리있는 NPC들은 `LightPhysicsUpdate`를 사용하여 비교적 가벼운 `ShapeCast`를 사용하여 위치와 속도 보정을 하게끔 최적화였다.

2.4 물리엔진 업데이트 오버헤드 최적화

모든 플레이어와 NPC같은 동적 객체들은 Kinematic Object인 `CharacterVirtual`을 사용하고 실제 게임의 모든 오브젝트들을 Static body로 생성하여서 update를 물리엔진에서 실행하는게 아닌 직접 update 함수를 래핑해서 위치와 속도를 넣어주고 `CharacterVirtual::Update`를 호출하여 그때의 물리엔진 업데이트를 수행하도록 최적화했다. 그러나 이렇게 kinematic으로만 물리Body를 만들어도 실제 게임서버 입장에서선 100개의 방의 100마리씩 NPC들은 10000개의 `ShapeCast`오버헤드가 계속 발생하는 것은 전체 서버의 성능이 다운되는 큰 병목이다.. 따라서 가벼운 물리 업데이트시 `ShapeCast`를 없애고 `RayCast`로 바꾼뒤 부딪힌 Body를 가져올때 어차피 모든 물리객체는 같은 스레드 안에 있으므로 `JPH::BodyLockInterfaceNoLock`를 사용하여 `Mutex` 오버헤드를 삭제하여 `BodyID`를 가져와서 충돌 체크 후 위치와 속도 조정만 하게끔 최적화하여 물리엔진 업데이트시 오버헤드를 획기적으로 줄였다.

2.4 정적 맵 데이터 메모리 공유 및 인스턴싱 최적화

MO게임 특성상 동일한 환경을 공유하는 수십~수백개의 인스턴스가 생긴다. 이때 각 인스턴스마다 물리 충돌체를 개별적으로 메모리에 적재하고 BVH 트리를 구성할 경우 막대한 메모리중복 할당과 심각한 방 생성 지연이 발생하므로 서버 부팅시 단 한번 맵데이터를 읽고 Mesh,Convex hull, HeightField등의 Shape들을 Jolt 내부 레퍼런스 카운터인 JPH::ShapeRef를 사용하여 서버 전역 데이터 메모리에 적재한다. 이제 새로운 인스턴스가 생성 될때 저장된 Shape를 참조해서 물리Body를 생성한다. 따라서 각 인스턴스의 물리객체 생성시 소요되는 오버헤드와 각 인스턴스의 메모리 소모량을 줄일수 있게 최적화했다.

2.5 스냅샷 리와인드를 활용한 지연보상 타격판정

네트워크 지연이 존재하는 환경에서는 클라이언트 화면의 위치와 서버의 위치간의 불일치가 필연적으로 발생한다. 이로 인해 유저가 정확히 몬스터를 조준하고 타격했음에도 서버에서는 허공에서 타격한것으로 보이는 문제가 발생한다. 스냅샷 기반 지연보상 아키텍처를 HitboxComponent에 도입했다. 맵 서버는 모든 객체의 위치를 일정 갯수를 과거 스냅샷 버퍼에 저장하고 클라이언트가 공격을 했을경우 타겟 NPC의 과거시점 스냅샷 트랜스폼 행렬을 역산해서 재구성한다. 이후 가상공간의 shape와 NPC 히트박스 간의 정밀한 교차를 통해 네트워크 지연환경에서도 100% 가까운 타격 판정의 일관성을 제공할

수 있다.

3. 결 론

3.1 개발 요약

본 연구에서는 IOCP 기반의 고성능 비동기 서버 아키텍처에 데이터 지향 설계(DOD) 기반의 Jolt Physics 라이브러리를 성공적으로 통합하여, 서버 권위적 물리 시뮬레이션이 가능한 MO 액션 게임 서버를 구현하였다.

멀티스레드 환경에서 발생할 수 있는 동기화 병목을 해결하기 위해 인스턴스(Room) 단위의 물리 공간 격리 및 로직 스레드별 메모리 할당기(TempAllocator) 재사용 구조를 설계하였으며, 대규모 NPC 스폰 시의 부하를 줄이기 위해 공간 분할(GridMap) 기반의 물리 LOD와 싱글스레드 기반 바디 접근 인터페이스를 통한 극한의 최적화를 수행하였다. 또한, 네트워크 지연 환경에서의 정밀한 판정을 위해 스냅샷 리와인드 기반의 지연 보상 시스템을 구축하여 액션 게임의 핵심인 타격 신뢰성을 확보하였다.

3.2 장점

고가용성 및 확장성: 전역 공간에서의 ShapeRef 공유 및 BVH 트리 재사용을 통해, 수백 개의 인스턴스가 생성되어도 메모리 사용량 증가를 최소화하고 방 생성 속도를 획기적으로 개선하였다.

CPU 오버헤드 최소화: NPC 간 충돌 레이어 컬링(Culling)과 정밀 시뮬레이션 대신 RayCast를 활용한 LightPhysicsUpdate

모드를 도입하여, 동일 사양 대비 수용 가능한 NPC 숫자를 극대화하였다.

보안 및 동기화 무결성: 서버 권위적 물리 모델을 통해 클라이언트의 변조(스피드해킹, 벽 뚫기 등)를 원천 차단하고, 서버 기반 캐릭터 컨트롤러(CCT)를 통해 모든 유저에게 동일한 물리적 결과값을 보장한다.

락 경합 제거: 로직 스레드 내부에서 BodyLockInterfaceNoLock을 사용함으로써 멀티스레드 환경의 고질적 문제인 Mutex 오버헤드를 제거하고 CPU 사이클을 로직 계산에 온전히 집중시켰다.

3.3 단점 및 개선점

물리 LOD의 정밀도 한계: 최적화를 위해 도입된 LightPhysicsUpdate의 RayCast 방식은 매우 가까운 지형이나 복잡한 좁은 틈새에서 캐릭터가 끼이거나 미세하게 떨리는 현상이 발생할 수 있다. 이를 위해 지형 경사도에 따른 보정 로직을 더욱 세밀화할 필요가 있다.

스냅샷 메모리 관리: 지연 보상을 위해 저장하는 과거 스냅샷 데이터의 양이 늘어날수록 서버의 메모리 부하가 증가한다. 스냅샷 저장 주기를 동적으로 조절하거나 델타 압축 기술을 도입하는 등의 개선이 필요하다.

복잡한 동적 장애물 처리: 현재는 정적 지형과 Kinematic 오브젝트 위주로 최적화되어 있으나, 실시간으로 파괴되거나 물리적으로 상호작용하는 대량의 동적 리지드 바디(Dynamic Rigid Body)가 투입될 경우에 대한 추가적인 부하 분산 연구가 요구된다.

3.4 향후 개발 방향

PPO 기반 AI 학습 통합: 구현된 물리 환경을 바탕으로 강화학습(Reinforcement Learning) 알고리즘을 연동하여, 지형지물을 물리적으로 인지하고 이용하는 지능형 NPC AI를 개발할 예정이다.

분산 서버 아키텍처 확장: 단일 프로세스 내의 Room 격리를 넘어, 물리 연산 부하가 큰 인스턴스를 별도의 물리 전용 서버 노드로 동적 할당하는 분산 물리 처리 시스템을 연구할 계획이다.

정교한 물리 기믹 추가: 현재의 타격 판정을 넘어, 유동체(Fluid)나 천(Cloth) 물리 시뮬레이션을 서버 권위적으로 처리하여 더욱 실감 나는 환경 상호작용을 제공하고자 한다.

4. 참고문헌

- [1] <https://github.com/jrouwe/JoltPhysics>
- [2] <https://jrouwe.github.io/JoltPhysics>

문제 해결을 위한 연락처

june25656@tukorea.ac.kr
ckswlsdl0916@gmail.com
pkj7323@tukorea.ac.kr

